



BRAIN FINANCE INC.

MVP Architecture and Technical Plan

The programmable bank and marketplace for AI agents

Version	2.0 — Crossmint Wallet Infrastructure + Expanded MVP Scope
Status	Confidential — Internal and Investor Distribution
Entity	Brain Finance Inc.
Website	brain.fi

CONFIDENTIAL — This document does not constitute an offer to sell securities.



1. Overview

Purpose of This Document

This document sets out the complete MVP architecture for Brain, a programmable bank and marketplace for AI agents. It covers the MVP scope and rationale, the full system architecture including all partner integrations, the account setup and agent deployment flow, the x402 payment execution flow, and the AI reasoning layer that powers agent intelligence. It is intended for investors, technical advisors, and the founding team.

What Brain Is

Brain is building the financial infrastructure layer for the agent economy. It combines a stablecoin neobank, an agent execution framework, and a developer marketplace into a single programmable financial environment. The platform operates at two levels:

Layer	Description
Brain Platform	Consumer-facing financial environment: stablecoin accounts, agent deployment, developer marketplace, debit card, trading, and p
Brain Protocol	Open infrastructure standard built on ERC-4337, ERC-8004, x402, and related standards. Independently deployable by any develo

What the MVP Must Prove

The investor demo loop is:

- 1. A user signs up
- 2. Deposits USDC
- 3. Creates an agent with a spend policy
- 4. The agent autonomously executes a payment to an external service via x402
- 5. The agent executes an on-chain trade via the integrated trading layer
- 6. A stablecoin payment is sent to an external recipient via the payments rail

The MVP does not need to be a full product. It needs to prove that the core loop works end to end: capital in, agent deployed, agent acts, policy enforces, payment settles on-chain, trade executes, receipt recorded.

2. MVP Scope

In Scope

- Stablecoin account (USDC deposit, balance, transfer) via Coinbase on-ramp
- Agent sub-account with isolated capital pool per agent
- Policy configuration: spend limit, allowed assets, time window, approval threshold
- BrainAccount smart contract (ERC-4337) deployed on Base
- PolicyValidator contract with on-chain policy proof verification
- ERC-8004 agent registry: agentId assignment, status, validation records
- x402 payment flow: parse 402 response, policy check, on-chain settlement, receipt
- One prebuilt agent template (payment agent, simplest flow)
- AI reasoning layer: Claude tool use, ReAct agent loop, three-tier memory
- Crossmint Embedded Wallets: user wallet provisioning, key management, USDC on-ramp
- Crossmint Agent Wallets: isolated agent execution wallets with programmable guardrails
- Trading integration — Hyperliquid: on-chain perpetuals and spot trading via agent
- Payments rail: stablecoin send/receive to external recipients, including x402 and direct transfer
- WireX bank account and card provisioning (runs in parallel, not blocking the demo)

Deferred to Phase 2

- IBAN accounts and fiat rails
- KYC integration (Sumsub)
- Lending and credit products (Aave, Compound)
- Agent swarms and multi-agent coordination
- Developer marketplace
- EIP-7702 EOA delegation
- Subscription billing (premium swarm tier)
- Polymarket prediction market integration

Timeline

Phase	What	Duration
Smart contracts and protocol	ERC-4337 account, PolicyValidator, ERC-8004 registry	TBD
Banking and on-ramp	USDC account, Coinbase on-ramp, WireX provisioning	TBD
Wallet infrastructure	Crossmint Embedded Wallets (user), Crossmint Agent Wallets (execution)	TBD
Trading layer	Hyperliquid integration, agent trade tool, policy guards	TBD
Payments rail	Stablecoin send/receive, x402 settlement, receipt indexer	TBD
AI reasoning layer	Claude tool use, ReAct loop, pgvector memory	TBD



Phase	What	Duration
Agent template and UI wiring	Policy config, dashboard, deploy flow	TBD
Testnet alpha	Internal demo, audit submission, bug fixes	TBD

3. System Architecture

Layer Overview

The MVP architecture is structured across seven distinct layers. Each layer validates independently while composing into a single deterministic execution pipeline. No action can bypass identity verification, policy enforcement, or payment validation.

Layer	Description
Client	Brain web app, mobile app, SDK. Users configure agent goals, review policy, monitor activity.
AI Reasoning	LLM reasoning (Claude / GPT-4o via tool use), ReAct agent loop (reason, act, observe), three-tier memory (context, vector, structured)
Identity	SIWX authentication for users and agents. ERC-8004 agent registry assigning persistent agentId.
Control Plane	Policy Engine (evaluate, sign proof), Agent Service (lifecycle, config), Payment Orchestrator (x402 flow, UserOp assembly), Trading
On-Chain	BrainAccount ERC-4337 smart account, PolicyValidator contract, Alchemy bundler (ERC-7769).
Partners	Crossmint Embedded Wallets (user wallet, on/off-ramp), Crossmint Agent Wallets (agent execution signing with programmable guardrails)
Infrastructure	Base (execution), Ethereum (identity anchor), Alchemy (RPC/bundler), pgvector (vector memory), Anthropic Claude + OpenAI back

Infrastructure Components

Component	Provider	Role
Execution network	Base	Primary chain for all agent transactions
Identity anchor	Ethereum	ERC-8004 registry, high-value settlement
RPC and bundler	Alchemy	UserOperation submission, ERC-7769 bundler
User wallets	Crossmint Embedded Wallets	User wallet provisioning, key management, USDC on-ramp. No vendor lock-in — signers rotate
Agent wallets	Crossmint Agent Wallets	Smart contract agent execution wallets with programmable guardrails, spend limits, and audit
Vector memory	pgvector (Postgres)	Short-term agent memory for recent observations
Trading	Hyperliquid	On-chain perpetuals and spot execution for trading agents
Payments	x402 + direct USDC transfer	Machine-to-machine and peer stablecoin payments
LLM API	Anthropic (Claude) + OpenAI back	Agent reasoning and tool use

4. Partner Integrations

Crossmint Embedded Wallets (User Wallets)

Crossmint Embedded Wallets owns the user side of the capital stack. It provisions the user's primary smart contract wallet on sign-up, handles private key management so Brain never touches user keys, and provides the on/off-ramp so users can deposit USDC from fiat without leaving the Brain app. Unlike EOA-based approaches, Crossmint wallets are ERC-4337 smart accounts with modular signer architecture — signers can be rotated without changing wallet addresses, eliminating vendor lock-in.

- Provisions user smart contract wallet at sign-up via email, passkey, or social login
- Manages private keys with no vendor lock-in — signers rotatable without address change
- Provides USDC on-ramp from fiat via headless, white-label flow
- Serves as the owner address of the BrainAccount smart contract
- Built-in AML screening, sanction list monitoring, and whitelabel KYC flows
- Chain-agnostic: same API covers EVM, Solana, Stellar for future expansion
- Gas sponsorship via native paymaster — no ETH required from users

Priority: Critical path. Crossmint Embedded Wallet must be live before BrainAccount deployment can be wired to a real user key. Initiate sandbox access in week 1.

Crossmint Agent Wallets (Agent Execution)

Crossmint Agent Wallets are non-custodial smart contract wallets designed specifically for autonomous agent execution. Each deployed agent receives its own Crossmint Agent Wallet as its execution address. These wallets enforce programmable guardrails directly in the smart contract — spend limits, allowlisted destinations, and approval thresholds are enforced on-chain, not just in application logic. This is architecturally superior to EOA-based agent wallets because a compromised or hallucinating agent cannot bypass the on-chain policy constraints.

- Each deployed agent receives its own Crossmint Agent Wallet as its execution address
- Signs UserOperations submitted to the ERC-4337 EntryPoint
- Isolated from the user's primary wallet by architecture and smart contract enforcement
- Programmable guardrails: spend limits, destination allowlists, velocity controls enforced on-chain
- Auditable transaction records with full accountability trail per agent
- Can only access capital explicitly allocated via BrainAccount authorizeAgent
- Signer rotation supported — agent wallets survive key compromise without address change

Priority: Critical path. Crossmint Agent Wallet must be live before any agent can sign a UserOperation. Initiate sandbox access in week 1, in parallel with Embedded Wallets. Note: Crossmint powers WireX's own wallet infrastructure, making this integration alignment strategically coherent.

Hyperliquid (Trading Layer)

Hyperliquid provides on-chain perpetuals and spot trading directly accessible from the agent execution layer. Trading is included in the MVP scope to demonstrate that Brain agents can autonomously manage financial positions — not just make payments. The trading agent type is the second prebuilt template alongside the payment agent.

- On-chain perpetuals and spot trading via Hyperliquid L1
- Agent submits signed orders via the Hyperliquid API, gated through the Policy Engine
- Per-trade and cumulative exposure limits enforced before order submission
- Trade receipts indexed alongside payment receipts in the unified activity ledger

- Separate allowAsset and maxPositionSize policy parameters for trading agents

Priority: MVP scope. Trading demo required for investor loop. Initiate Hyperliquid testnet access in week 1. Trading policy controls must be wired into PolicyValidator before demo milestone.

Payments Rail

Brain's payments layer combines x402 machine-to-machine settlement with direct USDC transfer for peer and business payments. Both flows run through the same Policy Engine and receipt indexer, giving Brain a unified payment ledger across agent-initiated and user-initiated transactions.

- x402 machine-to-machine: agent pays external HTTP services automatically
- Direct USDC transfer: user or agent sends to any wallet address or ENS
- Batch payments: single UserOperation settles multiple recipients
- Policy-gated: all payment intents evaluated before execution, signed proof required
- Receipt binding: every payment links HTTP request or transfer intent to on-chain tx hash
- Global reach: USDC on Base settles in seconds at near-zero cost

WireX (Banking and Card Layer)

WireX operates in the fiat and card layer, parallel to the crypto stack. It provisions the stablecoin account and IBAN account and issues the debit card. WireX's own wallet infrastructure is built on Crossmint smart wallets, creating direct alignment between Brain's wallet provider and banking partner.

- Provisions stablecoin account (USDC-denominated)
- Issues IBAN account for fiat connections
- Issues debit card linked to account balances
- Operates independently of the smart account and agent execution flow

Priority: Not blocking the investor demo. Initiate business agreement in week 1 but do not let it gate the demo milestone.

Integration Dependency Order

Integration	Blocks what	Priority
Crossmint Embedded Wallets	User sign-up, USDC deposit, BrainAccount owner address	Critical path, week 1
Crossmint Agent Wallets	Agent execution, x402 payments, trading, UserOp signing	Critical path, week 1
Alchemy RPC and bundler	All on-chain transactions	Critical path, week 1
Hyperliquid	Trading agent template, trade policy controls	MVP scope, week 1
Payments rail (x402 + USDC)	Payment agent template, receipt indexer	MVP scope, week 1
WireX bank and card	Card spending, IBAN rails	Non-blocking

5. Account Setup and Agent Deployment

Account Setup Flow

The account setup flow runs left to right, establishing the user's financial environment before any agent is deployed.

1	User registers via SIWX sign-in. Email and cryptographic signature create a scoped session.
2	Crossmint Embedded Wallet is provisioned. The user's primary smart contract wallet is created with key management handled by Crossmint. Users au
3	WireX account is provisioned. A stablecoin account and debit card are created in parallel.
4	BrainAccount smart contract is deployed on Base. The user's Crossmint wallet address becomes the owner.
5	User deposits USDC via the Coinbase on-ramp. Funds land in the BrainAccount.
6	Primary account is funded and ready for agent allocation.

Agent Deployment Flow

Agent deployment runs in parallel with account setup. Once both are complete, capital is allocated to the agent and it goes live.

7	User configures a policy: spend limit, allowed assets, time window, approval threshold. For trading agents: additional maxPositionSize and allowedMar
8	Policy hash is written on-chain via setPolicy on the BrainAccount.
9	A Crossmint Agent Wallet is created as the agent's execution address. Programmable guardrails are embedded at the smart contract level.
10	The agent is registered in the ERC-8004 registry. A persistent agentId is assigned.
11	The agent is authorised on the BrainAccount via authorizeAgent. It is now scoped to its policy.
12	Capital is allocated from the primary account to the agent's sub-account. The agent is live.

Key Architecture Principle: An agent can only act within the capital explicitly allocated to its sub-account. It has no visibility into and no access to the user's primary account or any other agent's allocation. This separation is enforced at the smart contract level by the Crossmint Agent Wallet's on-chain guardrails and the BrainAccount PolicyValidator — not by application logic.

6. x402 Payment Execution Flow

Overview

x402 is the protocol for machine-to-machine payments over standard HTTP. When an agent requests a paid resource, the server responds with a 402 Payment Required response and a structured payment header. Brain handles policy evaluation and on-chain settlement automatically. Agents and servers communicate only via standard HTTP headers.

Step-by-Step Flow

13	Agent sends GET /resource with Agent-ID header via its Crossmint Agent Wallet identity.
14	Server returns 402 with X-402-Payment header: amount, asset, merchant address, expiry, nonce.
15	Payment Orchestrator parses the header and constructs a PaymentIntent: agentId, resourceUri, amount, asset, merchant, expiry, nonce.
16	Policy Engine evaluates the PaymentIntent: budget check, asset allowlist, merchant allowlist, frequency limit. Issues a signed policy proof if approved.
17	UserOperation is assembled with the policy proof attached and submitted to the Alchemy bundler.
18	Alchemy bundler submits to the ERC-4337 EntryPoint. PolicyValidator verifies the proof on-chain. BrainAccount executes the USDC transfer via the Cr
19	Receipt Indexer records the result: agentId, PaymentIntent, on-chain tx hash, HTTP correlation ID.
20	ERC-8004 validation record is written to the agent registry.
21	Brain retries the original HTTP request with X-402-Receipt header. Server verifies and serves the resource.

Security Guarantees

Mechanism	What it Prevents
Nonces	Replay attacks on payment intents
Expiries	Stale or delayed charge execution
Per-merchant budgets	Overspending to a single counterparty
Crossmint Agent Wallet guardrails	Execution beyond on-chain spend limits, even if policy engine is bypassed
PolicyValidator (on-chain)	Execution without a valid signed policy proof
Receipt binding	Disputes: every payment links HTTP request to on-chain tx
Signer isolation	Agent wallet cannot access user wallet or other agents' allocations

7. Trading Execution Flow

Overview

The trading layer connects Brain agents to Hyperliquid's on-chain perpetuals and spot markets. Agents can autonomously execute trades as part of their objective loop, subject to the same policy framework that governs payments. No trade can execute without a valid signed policy proof from the Policy Engine.

Step-by-Step Flow

1	Agent reasoning layer (Claude tool use) calls <code>trade_hyperliquid()</code> with order parameters: asset, side, size, order type, price limit.
2	Trading Orchestrator constructs a <code>TradeIntent</code> : agentId, asset, side, size, price, timestamp.
3	Policy Engine evaluates the <code>TradeIntent</code> : <code>maxPositionSize</code> check, allowed markets check, cumulative exposure limit, cooldown window. Issues a signed policy proof.
4	Trading Orchestrator submits the signed order to Hyperliquid via its API, authenticated by the Crossmint Agent Wallet.
5	Hyperliquid executes the order on-chain. Fill confirmation returned.
6	Receipt Indexer records the trade: agentId, <code>TradeIntent</code> , Hyperliquid order ID, fill price, fill size, on-chain confirmation.
7	ERC-8004 validation record is written. Trade observation is stored in pgvector short-term memory for the next agent reasoning turn.

Trading Policy Parameters

Parameter	Description	Enforcement
<code>maxPositionSize</code>	Maximum notional per open position	Policy Engine + on-chain
<code>allowedMarkets</code>	Allowlist of tradeable asset pairs	Policy Engine
<code>maxDailyLoss</code>	Circuit breaker: halt trading if daily P&L exceeds threshold	Policy Engine
<code>cooldownWindow</code>	Minimum time between orders of same asset	Policy Engine
<code>cumulativeExposure</code>	Maximum total open exposure across all positions	PolicyValidator

8. AI Reasoning Layer

Recommendation: Model-Agnostic Runtime with Thin Abstraction

The recommended approach is to avoid hardcoding a single LLM provider into agent logic. Brain's value proposition is the financial infrastructure layer, not the AI layer. The intelligence must be pluggable. The recommended stack has four components.

Component 1: LLM Reasoning via Tool Use

Anthropic's Claude with tool use (function calling) is the recommended default. Tool use is the right primitive for financial agents: the LLM reasons about when to call Brain's execution functions rather than generating free-form text.

- Default: Claude via the Anthropic API with tool use
- Secondary: OpenAI GPT-4o (identical tool use patterns, same abstraction layer)
- The LLM proposes. The policy engine decides. The Crossmint Agent Wallet executes.

A hallucinating model, a jailbroken prompt, or a malicious agent template cannot bypass financial controls. The smart account and PolicyValidator enforce constraints independently of whatever the LLM decides. Crossmint Agent Wallet guardrails add a second enforcement layer at the smart contract level.

Component 2: ReAct Agent Loop

The agent loop is a lightweight custom implementation of the ReAct pattern: Reason, Act, Observe. This sits server-side in Brain's backend between the client request and the control plane.

1	LLM receives: agent objective, current account state, tool list, memory context.
2	LLM reasons and returns a structured tool call.
3	Loop passes the tool call to the Brain control plane as a proposed action.
4	Policy Engine evaluates. If approved: Crossmint Agent Wallet executes. If rejected: rejection reason returned to LLM as observation.
5	The result feeds back to LLM as an observation. Loop continues until objective is complete.

Component 3: Three-Tier Memory Architecture

Tier	Storage	Contents	Scope
In-context	LLM prompt window	Current objective, last few observations, active position	Current session only
Short-term	pgvector (Postgres)	Recent transaction embeddings, trade observations, conversation history	Retrieved by semantic similarity
Long-term structured	Agent database record	Strategy parameters, risk tolerance, persistent state, running totals	Persists across all sessions

Component 4: Tool Registry

Every function the LLM is allowed to call is declared as a JSON tool schema. Brain controls this list entirely.

Tool	What it Does
transfer()	Move USDC between accounts within policy limits
pay_x402()	Execute an x402 payment to an external service

Tool	What it Does
<code>trade_hyperliquid()</code>	Submit a signed order to Hyperliquid (perpetuals or spot)
<code>send_payment()</code>	Direct USDC transfer to a wallet address or ENS
<code>check_balance()</code>	Read current balance of the agent's sub-account
<code>get_policy()</code>	Retrieve the agent's active policy configuration
<code>get_positions()</code>	Retrieve current open trading positions and P&L
<code>record_action()</code>	Write an observation or decision to agent memory

MVP AI Configuration

For the investor demo, the following configuration is sufficient to demonstrate the full loop:

- Two agent types: payment agent and trading agent
- One LLM: Claude via the Anthropic API with five registered tools (`pay_x402`, `trade_hyperliquid`, `send_payment`, `check_balance`, `get_policy`)
- One memory store: pgvector table with the last 20 transactions/trades as short-term memory
- Payment demo target: an Anthropic or OpenAI API endpoint that accepts x402 headers
- Trading demo target: Hyperliquid testnet with a BTC-USDC perpetual position

This is enough to show an agent reasoning about whether to execute a payment or a trade, passing the decision through the policy engine, and settling on-chain via Crossmint Agent Wallet execution.